

NumPy Arrays in Two Dimensions

Shape and memory layout, indexing and slicing per axis, reductions along a named axis, reshape and length-one axes, locating a maximum in more than one dimension, broadcasting, and the matrix product.

01 Shape, and what it describes	05 Locating a maximum
02 Indexing and slicing per axis	06 Broadcasting
03 Reductions along an axis	07 The matrix product
04 Reshape and length-one axes	08 Loop to expression

Scope. Week 3 covered arrays with a single axis. Everything below needs at least two: naming an axis in a reduction, combining arrays whose shapes differ, and multiplying matrices. Boolean masks, coordinate arrays, counting and sorting are week 5 and are not repeated here.

After reading it you should be able to do the following without looking anything up.

- Predict the shape of a slice, and say which axes survive it.
- Reduce along a chosen axis and state the resulting shape before running it.
- Reshape an array, insert a length-one axis, and say whether the result is a view.
- Convert between a flat index and a per-axis one.
- Apply the broadcasting rules to two shapes and predict the result, or say why it is an error.
- Use the matrix product and distinguish it from elementwise multiplication.

Every example assumes `import numpy as np`.

01

Shape, and what it describes

The elements of an array sit in one line in memory whatever the shape says. The shape is the rule for turning a multidimensional index into a position in that line.

```
m = np.array([[2, 3, 4], [1, 5, 8]])

m.shape      # (2, 3)    two rows, three columns
m.ndim       # 2        number of axes
m.size       # 6        total elements, the product of shape
len(m)       # 2        length along axis 0 only, not the total
```

Axis 0 runs down the rows and axis 1 runs across the columns. For an array of three axes with shape `(2, 3, 4)` the convention continues: axis 0 selects one of the two sheets, axis 1 a row within it, axis 2 a column.

ROW-MAJOR ORDER

Elements are laid out with the **last axis varying fastest**. A `(3, 2)` array runs `[0,0]` `[0,1]` `[1,0]` `[1,1]` `[2,0]` `[2,1]` through memory. That order is what `reshape` reads and writes, and what section 5 depends on.

Because the shape is only a description of a flat block, changing it moves nothing. Transposing does not move anything either: it reverses the axis order and leaves the data untouched, which is why `m.T` is a view.

```
m.T.shape          # (3, 2)
m.T.base is m      # True    a view, no data copied
```

02

Indexing and slicing per axis

Multidimensional indexing takes one entry per axis, separated by commas. Whether an axis survives depends on how it was indexed.

```
b = np.array([[0, 1, 2], [3, 4, 5]])    # shape (2, 3)

b[1, 2]      # 5          a single index on every axis gives a scalar
b[1]         # [3 4 5]    same as b[1, :], trailing colons may be dropped
b[1:, :]     # [[3 4 5]] every row but the first
b[:, 1:]     # [[1 2]]    every column but the first
b[:, 1:]     # [4 5]]
b[... , 0]   # [0 3]      ... stands for as many full axes as needed
```

A SINGLE INDEX REMOVES AN AXIS, A SLICE KEEPS IT

This is the rule that decides every resulting shape, and it holds even when the slice selects exactly one element.

```
b[1, :].shape    # (3,)    axis 0 indexed away
b[1:2, :].shape  # (1, 3)  axis 0 sliced, so it stays, with length 1
b[:, 1].shape    # (2,)    axis 1 indexed away
b[:, 1:2].shape  # (2, 1)  axis 1 sliced, so it stays
```

The same applies at higher rank. For `x` of shape `(2, 3, 4)`, `x[:, 1]` has shape `(2, 4)` while `x[:, 1:2]` has shape `(2, 1, 4)`.

A slice along any axis is still a **view**, exactly as in one dimension, so writing through it changes the original. An out-of-range slice is clipped; an out-of-range index raises.

TWO BRACKETS ARE NOT ONE BRACKET

`b[1][2]` and `b[1, 2]` give the same answer here, but they do different work. The first builds the intermediate row `b[1]` and then indexes it; the second goes straight to the element. The difference matters when the first index is a slice, because `b[:, 1]` and `b[:, 1][1]` are not the same thing at all: `b[:, 1]` is the whole array, so `b[:, 1][1]` is row 1.

03

Reductions along an axis

With no argument a reduction collapses the whole array to one number. The `axis` argument narrows that to one direction.

THE AXIS YOU NAME IS THE AXIS THAT DISAPPEARS

This is easier to apply than "sum the rows" or "sum the columns", which are ambiguous in English and reliably get swapped. Check the shape rather than the wording.

```
m = np.array([[2, 3, 4], [1, 5, 8]])      # shape (2, 3)

m.sum()                # 23                everything collapses, a scalar
m.sum(axis=0)          # [ 3  8 12]        axis 0 gone, shape (3,): one total per column
m.sum(axis=1)          # [ 9 14]          axis 1 gone, shape (2,): one total per row
```

If the input has shape `(2, 3)` and you name `axis=0`, the result has shape `(3,)`, and there is only one thing that can mean. The rule extends to any rank: for `x` of shape `(2, 3, 4)`, summing along axis 0, 1 and 2 gives shapes `(3, 4)`, `(2, 4)` and `(2, 3)`.

Every reduction from week 3 takes `axis`: `sum`, `prod`, `mean`, `min`, `max`, `std`, `var`, `any`, `all`, `argmin`, `argmax`.

KEEPDIMS, AND WHY IT EXISTS

A reduction removes the axis, which makes the result awkward to combine with the original.

`keepdims=True` leaves the axis in place with length 1, and that shape broadcasts back against the input (section 6).

```
m.sum(axis=1).shape      # (2,)
m.sum(axis=1, keepdims=True).shape  # (2, 1)

m - m.mean(axis=1, keepdims=True)  # subtract each row's mean from that row
m - m.mean(axis=1)                 # ValueError: shapes (2,3) and (2,)
```

04

Reshape and length-one axes

Reshaping reinterprets the same block of memory under a new index rule. The product of the new shape must equal `size`.

```
b = np.arange(6)          # [0 1 2 3 4 5]          shape (6,)
b.reshape(3, 2)           # [[0 1]
                          #  [2 3]
                          #  [4 5]]          shape (3, 2), same memory
b.reshape(-1)             # flat again, whatever the input shape was
b.reshape(-1, 1).shape    # (6, 1)              a column
b.reshape(2, -1).shape    # (2, 3)

b.reshape(4, -1)
# ValueError: cannot reshape array of size 6 into shape (4,newaxis)
```

One axis may be written as `-1`, meaning "whatever makes the total come out right". Only one, since two unknowns have no unique solution.

A reshape returns a view whenever the existing layout allows it, and copies when it does not.

`np.arange(6).reshape(3, 2)` is a view; `m.T.ravel()` is a copy, because reading a transposed array in row-major order does not correspond to any stride pattern over the original. Treat a reshape as something to read from, and call `.copy()` when you need a guarantee either way.

LENGTH-ONE AXES

Indexing with `None`, also spelled `np.newaxis`, inserts an axis of length 1 at that position. Nothing is copied and no element changes, only the shape.

```
b.shape           # (6,)
b[None, :].shape  # (1, 6)    a row,    same as b.reshape(1, -1)
b[:, None].shape  # (6, 1)    a column, same as b.reshape(-1, 1)
```

A length-one axis looks pointless in isolation. It is how you state which way a one-dimensional array should be oriented when it meets a two-dimensional one, which is what section 6 is about.

05

Locating a maximum

On an array of more than one axis, `argmax` does not return the index you expect.

```
m = np.array([[2, 3, 4], [1, 5, 8]])    # the maximum, 8, sits at [1, 2]

m.max()           # 8        the value
m.argmax()        # 5        NOT (1, 2): the sixth element in memory
m.reshape(-1)[5]  # 8        correct, if you flatten first
```

`argmax` returns a single index into the flattened array, in the row-major order of section 1.

`np.unravel_index` converts that back into per-axis indices, and `np.ravel_multi_index` converts the other way. Both accept arrays of indices, not only single ones.

```
np.unravel_index(5, m.shape)           # (1, 2)
m[np.unravel_index(m.argmax(), m.shape)] # 8
```

If what you want is one answer per row, name the axis, and the rule from section 3 applies unchanged: the axis you name is the one that disappears.

```
m.argmax(axis=1)    # [2 2]    the winning column within each row
m.argmax(axis=0)    # [0 1 1]  the winning row within each column
```

A POSITION INSIDE A SLICE IS A POSITION INSIDE THAT SLICE

Taking `argmax` of `grid[:, c0:c1]` gives a flat index into the slice, whose width is `c1 - c0`, not into `grid`. Converting it back needs both the slice's own shape and its offset:

```
band = grid[:, c0:c1]
r, c = np.unravel_index(band.argmax(), band.shape)
row, col = r, c + c0          # c was measured from the band's left edge
```

Ties are broken by position: the first occurrence in memory wins, always.

Broadcasting

Elementwise operations require equal shapes. Broadcasting is the set of rules under which unequal shapes are treated as equal, by repeating elements virtually rather than copying them.

THE RULE

Line the two shapes up **from the right**. For each position, the lengths must either be equal, or one of them must be 1. A missing axis on the left counts as 1. The result takes the larger length in each position.

```
(2, 3, 4) and (4,) -> (2, 3, 4)    the (4,) aligns with the last axis
(2, 3, 4) and (3, 1) -> (2, 3, 4)    the 1 stretches to 4
(240, 360) and (360,) -> (240, 360)  one value per column
(3,) and (3, 1) -> (3, 3)           both stretch, an outer product shape
(2, 3) and (3, 2) -> error
```

```
np.zeros((2, 3)) + np.zeros((3, 2))
# ValueError: operands could not be broadcast together with shapes (2,3) (3,2)
```

Alignment from the right is the part to remember. A `(360,)` vector against a `(240, 360)` grid gives one value per **column**, applied to every row, with no reshaping. To apply one value per **row** instead, the vector has to be turned into a column first, with `[:, None]`.

```
grid - offsets          # offsets is (360,): one per column
grid - offsets[:, None] # offsets is (240,): one per row, now shape (240, 1)
```

BROADCASTING IS FREE, THE RESULT IS NOT

No copy is made of the stretched operand, so broadcasting itself costs nothing. The *result* is a real array, and it has the broadcast shape. Comparing every element of an array against every other is the common case where this matters:

```
a = np.arange(10_000)
(a[:, None] > a[None, :]).sum(axis=1)    # correct, and allocates 100 million booleans
```

That expression replaces a double loop and is far faster, but it trades time for memory, and at `n = 100_000` it would ask for 10 GB. Week 5 solves the same problem with a sort and a binary search, in $O(n \log n)$ and no square array.

The matrix product

A different operation from elementwise multiplication, with a different shape rule.

```
L = np.arange(6).reshape(2, 3)    # (2, 3)
R = np.arange(12).reshape(3, 4)   # (3, 4)

L @ R          # shape (2, 4)      np.matmul(L, R), or L.dot(R)
L * R          # ValueError: shapes (2,3) and (3,4) not broadcastable
```

For `(m, k) @ (k, n)` the **inner** dimensions must agree and the result is `(m, n)`. Entry `[i, j]` is the sum over the shared axis of row `i` of the left operand against column `j` of the right. The order matters:

`L @ R` and `R @ L` are different operations and usually only one of them is even defined.

If the shapes do not line up, transposing one operand will often make the error go away while computing something you did not intend. Check which product you meant before reaching for `.T`.

`@` also handles the one-dimensional cases: a `(n,)` against a `(n,)` is the inner product and gives a scalar, and a `(m, n)` against a `(n,)` gives `(m,)`.

08

Loop to expression

Common nested-loop forms and their array equivalents. Each expression in the right-hand column runs in compiled code and references no index variable.

WHAT THE LOOP DOES	AS AN ARRAY EXPRESSION
Total of each row	<code>arr.sum(axis=1)</code>
Total of each column	<code>arr.sum(axis=0)</code>
The largest row total	<code>arr.sum(axis=1).max()</code>
Sum inside a rectangle	<code>grid[r0:r1, c0:c1].sum()</code>
Subtract each row's mean from that row	<code>arr - arr.mean(axis=1, keepdims=True)</code>
Subtract one value per column	<code>grid - offsets</code>
Subtract one value per row	<code>grid - offsets[:, None]</code>
Winning column in each row	<code>arr.argmax(axis=1)</code>
Position of the overall maximum	<code>np.unravel_index(arr.argmax(), arr.shape)</code>
Compare every element against every other	<code>a[:, None] > a[None, :]</code>
Matrix product	<code>L @ R</code>

SHAPE CHECKLIST

Most errors at this level are shape errors, and they are all predictable before running anything.

EXPRESSION	RESULT SHAPE
<code>arr[i, j]</code>	scalar, both axes indexed away
<code>arr[i], arr[:, j]</code>	one axis removed
<code>arr[i:i+1], arr[:, j:j+1]</code>	rank unchanged, that axis has length 1
<code>arr.sum(axis=k)</code>	axis <code>k</code> removed
<code>arr.sum(axis=k, keepdims=True)</code>	axis <code>k</code> becomes length 1
<code>arr.T</code>	axis order reversed, a view
<code>arr[:, None]</code>	a length-one axis inserted, a view
<code>(m, k) @ (k, n)</code>	<code>(m, n)</code>

FURTHER READING

- [Broadcasting](#), the reference for section 6, with more worked shape pairs.
- [Indexing on ndarrays](#). The basic indexing section covers sections 2 and 4.
- [Linear algebra](#), for what exists beyond the matrix product itself.
- [The routine reference](#). Most array code people write by hand already exists there under a name.

SUMMARY

Shape is the rule for reading one flat block of memory, and the last axis varies fastest. A single index removes an axis and a slice keeps it. A reduction removes the axis it is given, and `keepdims=True` leaves it at length 1 so the result broadcasts back. `argmax` on more than one axis returns a flat index, which `np.unravel_index` converts. Broadcasting aligns shapes from the right and requires each pair of lengths to be equal or one of them to be 1. And the matrix product needs the inner dimensions to agree, which is a different rule from elementwise multiplication.

Not in this document. Boolean masks across several axes, coordinate arrays from `np.nonzero` and `np.argwhere`, indexing with integer arrays on more than one axis, counting with `np.unique` and `np.bincount`, and sorting and searching. Those are week 5.